

VMUD: Detecting Recurring Vulnerabilities with Multiple Fixing Functions via Function Selection and Semantic Equivalent Statement Matching

Anonymous Author(s)*

ABSTRACT

The widespread use of open-source software (OSS) has led to extensive code reuse, making vulnerabilities in OSS significantly pervasive. The vulnerabilities due to code reuse in OSS are commonly known as vulnerable code clones (VCCs) or recurring vulnerabilities. Existing approaches primarily employ clone-based techniques to detect recurring vulnerabilities through matching vulnerable functions in software projects. These techniques do not incorporate specially-designed mechanisms for vulnerabilities with multiple fixing functions (VM). Typically, they generate a signature for each fixing function and report VM using a matching-one-in-all approach. However, the variation in vulnerability context across diverse fixing functions results in varying accuracy levels in detecting VMs, potentially limiting the effectiveness of existing methods.

In this paper, we introduce VMUD, a novel approach for detecting Vulnerabilities with Multiple Fixing Functions (VM). VMUD identifies vulnerable function clones (VCCs) through function matching similar to existing methods. However, VMUD takes a different approach by only selecting the critical functions from VM for signature generation, which are a subset of the fixing functions. This step ensures VMUD to focus on fixing functions that offer sufficient knowledge about the VM. To cope with the potential decrease in recall due to excluding the remaining fixing functions, VMUD employs semantic equivalent statement matching using these critical functions. It aims to uncover more VM by creating two signatures of each critical function and match precisely by contextual semantic equivalent statement mapping on the two signatures. Our evaluation has demonstrated that VMUD surpasses state-of-the-art vulnerability detection approaches by 17.6% in terms of F1-Score. Furthermore, VMUD has successfully detected 275 new VM from 84 projects, with 42 confirmed cases and 5 assigned CVE identifiers.

1 INTRODUCTION

In recent years, the open-source software (OSS) ecosystem has witnessed substantial growth [4, 13], providing developers with a vast array of options for using free software. It has become a common practice for developers to adopt third-party OSS components to accelerate development and reduce costs [24]. Consequently, the vulnerability landscape has expanded beyond traditional proprietary software to encompass open-source software (OSS). The increasing vulnerabilities in open-source software have raised significant concerns, particularly regarding supply chain attacks [7]. Attackers can exploit these vulnerabilities, affecting not only the open-source software itself but also its downstream users, thus amplifying potential financial gains and cyber chaos [2].

Notably, downstream software shares or reuse logic from upstream OSS components. As these OSS components evolve and downstream software undergoes customization, the code initially

derived from OSS components diverges from the original one at both ends. Consequently, it leads to challenges in mitigating *Vulnerable Code Clones* (VCCs) [37]. When facing incompatible changes or customized OSS components, mitigating VCCs by replacing OSS components with newer versions becomes infeasible or cumbersome, especially in languages like C/C++ that lack a dominant package manager.

In this context, developers often resort to manually finding and backporting security patches into downstream software. However, this process is time-consuming and resource-intensive. Notably, 80% of attacks leveraged the vulnerabilities reported three or more years ago [3], which indicates that the issue of VCCs has not been fully resolved.

Typically, developers employ vulnerable code clone (VCC) discovery techniques [23, 49, 50, 54] to identify reused components (such as files, functions, or fragments) in a target project program. They can also use learning-based methods for classifying vulnerable functions [6, 8, 38, 44, 58]. However, the effectiveness of vulnerability detection is often hindered in cases of *Vulnerabilities with Multiple Fixing Functions* (VM), where fixing patches span across multiple functions or files. Unfortunately, VM are inevitably prevalent in real world due to complex inter-procedural program logic, posing challenges for accurate VM detection.

Limitations of Existing Approaches. Existing research often addresses the detection of *Vulnerabilities with Multiple Fixing Functions* (VM) by dividing the task into matching vulnerability characteristics within individual functions (*i.e.*, matching-one-in-all approach). Clone-based techniques [23, 49, 50, 54] identify vulnerable code clones (VCCs) within functions, while learning-based approaches represent functions in abstract spaces and classify them as vulnerable or not [6, 8, 38, 44, 58]. However, these approaches assume equal importance for all fixing functions in a vulnerability, potentially leading to over-representation resulting in false alarms. Additionally, while software composition analysis (SCA) techniques [19, 49, 52, 53, 55] can detect VM, they are primarily designed for identifying reused OSS components, which may overlook specific vulnerability or fixing characteristics, resulting in false alarms.

Challenge. Naturally thinking, assigning all fixing equal importance is straightforward and may not accurately represent the actual characteristics of vulnerabilities. Some fixing functions may carry adequate information to represent a vulnerability, while others may not. However, it is challenging to measure and weigh the unique fingerprints of each fixing function to a vulnerability. The primary challenge lies in how to capture and distinguish the uniqueness of fixing functions and identify the critical ones for a vulnerability.

Our Approach. To tackle this challenge, we introduce VMUD, a novel approach for Vulnerabilities-with-Multiple-Fixing-Function-Detection. Unlike traditional methods that treat all fixing functions equally, VMUD prioritizes the fixing functions and selects the most

critical functions for vulnerability representation (see Section 3.3.1). This selective approach involves matching the signatures [23, 54] of these critical functions. To cope with the potential decrease in recall due to excluding the remaining fixing functions, we employ semantic equivalent statement matching, which consists of program rephrasing [43] (see Section 3.3.2) and contextual semantic equivalent statement mapping (see Section 3.4.3). The program rephrasing aims to uncover more VM by creating another signature for the rephrased form of the critical functions, while contextual semantic equivalent statement mapping aims to match the composing statements precisely based on their contextual semantic equivalence.

Evaluation. We implemented VMUD and generated vulnerability signatures from 810 CVEs. We conducted an evaluation using a dataset comprised of the top 1,000 C/C++ real-world projects selected from GitHub. Comparing VMUD with state-of-the-art techniques, VMUD achieves a f1-score of 0.84, outperforming the best state-of-the-art at 17.6%. VMUD discovered 275 VM across 104 projects, with 42 of them confirmed by developers and 5 of them assigned with corresponding new CVE identifiers. We observed the robustness of VMUD regarding the number of fixing functions in the VM. We conducted an ablation study to observe the contribution of key components in VMUD. We also measured the threshold sensitivity and performance of VMUD. The results demonstrate VMUD's effectiveness and efficiency in identifying VM within real-world projects.

Contribution. We summarize our contributions as follows:

- We propose VMUD, a novel approach to detect Vulnerabilities with Multiple Fixing Functions (VM) in real-world projects.
- We evaluate VMUD on our ground truth collected from real-world C/C++ projects and demonstrate its effectiveness compared with the state-of-the-art approaches.
- VMUD has discovered 275 vulnerabilities in 84 C/C++ projects, 42 of which are confirmed by developers, 5 has been assigned with CVE identifiers, demonstrating its effectiveness in real-world projects.

2 MOTIVATION

We clarify the problem of detecting Vulnerabilities with Multiple Fixing Functions (VM) and discuss examples that motivate our design of VMUD.

2.1 Problem

Vulnerabilities with Multiple Fixing Functions (VM). We concentrate our goal on detecting recurring vulnerabilities with multiple fixing functions (VM) in the open source software (OSS). Specifically, developers need to apply fixing changes in vulnerable code to fix vulnerabilities and release new safe versions. The fixing action for vulnerabilities follows the same routine as other actions in evolutionary software, such as general bug fixing, feature addition, and code refactoring. They usually record the corresponding code changes by Version Control Tools (VCS). Due to the software becoming more complex, developers usually decouple program logic [42] to facilitate its flexibility. Thus, the code changes are complex, and distributed across multiple locations. e.g., classes, functions, etc. They may have changed patterns [56], tangled changes [18] (i.e.,

Table 1: Fixing functions of CVE-2021-32134 and FPs in the target project matched by VUDDY, V1SCAN and VMUD

Fixing File	Fixing Functions	VUDDY	V1SCAN	VMUD
<i>box_code_base.c</i>	<i>mp4s_box_new</i>	1	1	0
<i>box_code_base.c</i>	<i>encs_box_new</i>	1	1	0
<i>sample_desc.c</i>	<i>gf_isom_sample_entry_init</i>	0	0	0
<i>media.c</i>	<i>Media_GetESD</i>	0	0	0

unrelated or loosely related code changes), or systematic changes [22]. Taking the Big-Vul[11] dataset for example, it is widely used for vulnerability detection by many research works [41, 45, 57]. The data is constructed by crawling the public Common Vulnerabilities and Exposures (CVE) database and CVE-related source code repositories, comprising 3,754 code vulnerabilities and containing 11,823 vulnerable functions. We can observe that average fixing a vulnerability involves three fixing functions. Therefore, we focus on the problem of VM.

Due to the reason that developers fix vulnerabilities and conduct other evolutionary actions in the same software, their symptoms are similar. Code changes for fixing Vulnerabilities also exhibit various patterns such as change patterns, tangled changes, or systematic changes. Consequently, different fixing functions contribute unequally in terms of importance and characteristics to identify the vulnerability. However, existing research often treats each function equally [23, 49, 50, 54], triggering a vulnerability alarm if any of the functions match. Applying uniform importance to all fixing functions would result in diverse effectiveness per function. i.e., some functions may carry adequate information to represent a vulnerability, resulting in high accuracy, while others may not. Therefore, it is not reasonable to assign equal importance to all fixing functions. Li et al. [28] investigated inter-procedural vulnerabilities, where the statements to be patched and the statements triggering the vulnerability belong to different functions. Their approaches are based on a carefully designed list of vulnerability-trigger statements regarding popular CWE types, hindering its effectiveness in a wide range of vulnerability types.

Challenges. It is challenging to recognize the functions that contribute higher accuracy against the functions that contribute lower accuracy about one vulnerability. The key problem lies in measuring and weighing the unique fingerprints of each fixing function so that we can prioritize the functions. We refer to the recognizing task as *selecting critical functions* and filtering out less important functions. However, discarding the less important functions would result in a drop in recall as it would miss some vulnerabilities discovered by these functions. Therefore, we enhance the signature generation and matching by semantic equivalent statement matching to cope with the recall drop and improve our detection accuracy.

2.2 Motivating Example

Vulnerabilities with Multiple Fixing Functions (VMs). We illustrate our motivation using CVE-2021-32134[17, 34] from the *gpac/gpac* project. This vulnerability involves four fixing functions distributed across three files (see Table 1). i.e., *mp4s_box_new* and *encs_box_new* in *box_code_base.c*, *gf_isom_sample_entry_init* in *sample_desc.c*, and *Media_GetESD* in *media.c*. We also present the

```

1 GF_Box *mp4s_box_new(){
2   ISOM_DECL_BOX_ALLOC(GF_MPEGSampleEntryBox, GF_ISOM_BOX_TYPE_MP4S);
3   gf_isom_sample_entry_init((GF_SampleEntryBox*)tmp);
4   - tmp->internal_type = GF_ISOM_SAMPLE_ENTRY_MP4S;
5   return (GF_Box *)tmp;
6 }
7 GF_Box *encs_box_new(){
8   ISOM_DECL_BOX_ALLOC(GF_MPEGSampleEntryBox, GF_ISOM_BOX_TYPE_ENCS);
9   gf_isom_sample_entry_init((GF_SampleEntryBox*)tmp);
10  - tmp->internal_type = GF_ISOM_SAMPLE_ENTRY_MP4S;
11  return (GF_Box *)tmp;
12 }

```

(a) box_code_base.c

```

1 void gf_isom_sample_entry_init(GF_SampleEntryBox *ent){
2   + ent->internal_type = GF_ISOM_SAMPLE_ENTRY_MP4S;
3 }

```

(b) sample_desc.c

```

1 GF_Err Media_GetESD(GF_MediaBox *media, u32 sampleDescIndex, GF_ESD
2 **out_esd, Bool true_desc_only) {...
3   switch (type) {
4   case GF_ISOM_BOX_TYPE_MP4V:
5   +   if (entry->internal_type != GF_ISOM_SAMPLE_ENTRY_VIDEO)
6   +   return GF_ISOM_INVALID_MEDIA;
7   ESDa = ((GF_MPEGVisualSampleEntryBox*)entry)->esd;
8   if (ESDa) esd = (GF_ESD *) ESDa->desc; ...
9   }...
10  if (*out_esd == NULL) return gf_odf_desc_copy((GF_Descriptor
11  *)esd, (GF_Descriptor **)out_esd); ...

```

(c) media.c

Figure 1: A Motivating Example of Patches from *gpac/gpac* for Fixing CVE-2021-32134

corresponding code and patches in Figure 1. Notably, although there are statement deletions in *mp4s_box_new* and *encs_box_new* (Figure 1(a)), these deletions do not alter the overall semantics as the deleted statements are reintroduced in *gf_isom_sample_entry_init* (Figure 1(b)). We employed state-of-the-art approaches, including VUDDY [23], MVP [54], MOVERY [50], and V1SCAN [49] to detect VCCs regarding this vulnerability. While MVP and MOVERY did not yield false positives, both VUDDY and V1SCAN reported two false positives in *gpac/gpac* [16] in Table. 1 (Note that we also consider VCCs within the same but different versions of the project.). Comparatively, our tool VMud avoids false positives by exclusively considering *gf_isom_sample_entry_init* and *Media_GetESD* as critical functions using Critical Function Selection in Signature Generation (see Section 3.3.3).

Additionally, we evaluate the efficacy of utilizing our critical functions as vulnerability signatures (see Section 4) and apply them specifically to MVP [54] and V1SCAN [49]. Notably, we did not apply VUDDY [23] and MOVERY [50] due to that both tools did not provide the matched vulnerable functions from the original VM in their generated output. The outcomes are detailed in Table. 2. The analysis reveals drops in false positives (FPs) by 119 for MVP and 18 for V1SCAN when employing the critical function signatures. Although true positives (TPs) decrease by 39 for both tools, the precision in Changed TPs and FPs are lower than the original precision for each tool. This underscores the effectiveness of our critical function selection in reducing false positives and enhancing precision, despite the trade-off resulting in more true positives converting to false negatives. To address this trade-off and improve recall while using the critical functions, we employ the program Semantic-Centered Statement Matching in our approach. i.e., using *Program Rephrasing* and *Contextual Equivalent Statements Map* (see Section 3).

Table 2: Number of True Positives (TP), False Positives (FP), False Negatives (FN) Changed Using Critical Function in MVP and V1SCAN. (Precision_Δ denotes the Precision in Changed TPs and FPs while Precision_o Denotes the Original Precision without using Critical Functions)

Tool	Numbers Changed (#)			Precision _Δ	Precision _o
	TP	FP	FN		
MVP	-39	-119	+39	0.24	0.81
V1SCAN	-39	-18	+39	0.68	0.71

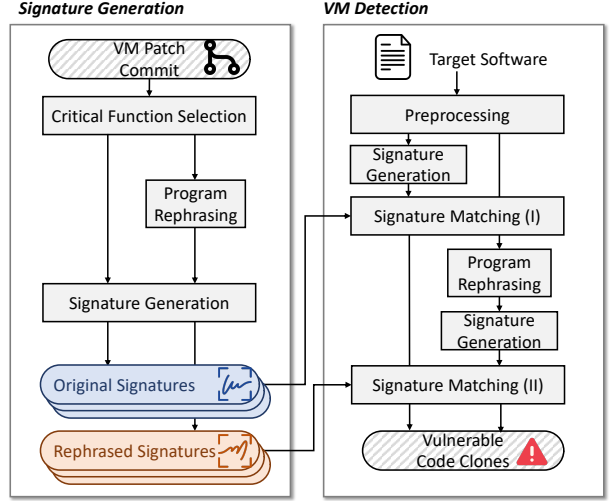


Figure 2: The Overview of VMud

3 DESIGN OF VMUD

In this section, we present the design of VMud, a novel approach to detecting Vulnerable Code Clones (VCCs) with Multiple Fixing Functions (VM). We first provide an overview of VMud and illustrate the key techniques in our paper. Then, we clarify the terminology and definition throughout the section. Finally, we present the technical details of Signature Generation and VM Detection.

3.1 Overview

VMud differs from existing works in two aspects. On the one hand, existing works don't distinguish the different importance of functions in VM, thus hindering the effectiveness of detecting VM. We select the critical functions and particularly enhance the signatures for these functions. On the other hand, existing approaches have imperfect matchings of statements. The reason is that most approaches only consider the syntactical equality of the statements (e.g., using `=`, `==` or `strcmp`) instead of the semantic equality (i.e., the equality of the surrounding contexts of the statements). Differently, VMud first prioritizes and selects the critical functions among multiple fixing functions during Signature Generation (see Section 3.3). Then, VMud designs a new semantic-centered statement matching technique (see Section 3.4) to redefine the equality of the statements from the target function and vulnerable functions. Additionally, VMud leverages program rephrasing [43] (see Section 3.3) to create semantically equivalent transformations of the expression, thus creating new rephrased signatures for the critical functions. It allows VMud to filter out false positives introduced

by less important functions and imperfect matching, and find true positives using the rephrased signature.

Figure 2 illustrates the workflow of VMUD, comprising two main stages: Signature Generation (see Section 3.3) and VM Detection (see Section 3.4). In the Signature Generation stage, VMUD takes a VM patch commit as input and extracts the fixing functions. Subsequently, it identifies critical functions among these fixing functions. Then, the critical functions either undergo the Abstraction and Normalization directly or the Program Rephrasing [43] beforehand. Next, VMUD extracts the original and rephrased signatures of these critical functions. In the VM Detection stage, VMUD reduces the detection space by preprocessing the target project, similar to existing works [49, 50]. Subsequently, the preprocessed software progresses through the Signature Matching (I) and Program Rephrasing. If necessary, the rephrased program enters the Signature Matching (II). Finally, VMUD outputs the identified vulnerable code clones.

3.2 Terminology and Definition

- **Vulnerable Code Clones (VCCs):** The vulnerable code clones [23] are reused OSS components in the target project program that contain vulnerabilities. Typically, the granularity of VCCs can vary at the component, file, function, or fragment level. We focus on detecting the general type of vulnerabilities which has multiple fixing functions. Therefore, the granularity of VCCs in our context can be a component, multiple files, multiple functions, or multiple fragments in multiple functions.
- **Critical Function:** In VM, the different fixing functions contribute diverse contexts for the vulnerability. The critical functions should carry the most significant fixing information to represent the vulnerability. We denote the critical functions as $\mathcal{F}$.
- **Signature.** The signatures for functions $\mathcal{F}$ are denoted as $\mathcal{X}$. Each entry X in the $\mathcal{X}$ corresponds to the signature for a function $f \in \mathcal{F}$. X_{org} denotes the original signature and X_{rep} denotes the rephrased signature for f . We use X^P to denote the vulnerability signature and X^V to denote the patched signature. For a target project function, we use X^T to denote the signature. X is a 3-tuple $\langle Syn, Sem, See \rangle$ where Syn consists of the syntactic statements of the vulnerable (resp. patched) function, Sem represents the semantic relationships of the vulnerable (patch) function, See represents the Semantic Enhanced (See) signature of the vulnerable (resp. patched) function.
- **Rephrased Program.** Program Rephrasing is a type of Program Transformation [43]. It involves changing one program into another, a practice widely used in software engineering for tasks like compilation, optimization, refactoring, and program synthesis. Program transformation can be split into program translation, where source and target languages differ, and program rephrasing, where both languages are the same. In our case, we employ program rephrasing to unify the “idioms” in vulnerable code and the target project, thus improving the function’s syntactic consistency.

3.3 Signature Generation

We gather the patches, which are added lines, and deleted lines, denoted as C_{add} and C_{del} , correspondingly, as well as their corresponding fixing commit ID for the given CVE VM patch commit. We

generate the original signatures X_{org} and the rephrased signatures X_{rep} for VM.

VMUD starts by giving an input as a CVE ID and traces patches to find the vulnerability-fixing commit in two ways. First, it searches the CVE ID in open-source software repositories’ histories with vulnerability keywords and filters out irrelevant commits. The irrelevant commits include reverted commits, merging commits, etc. Second, it leverages the National Vulnerability Database (NVD) [35] reference pages for silent fixes (i.e., commits without explicit CVE ID mentions). This database provides valuable metadata and referencing pages related to CVE fixes. Combining the two approaches, we gather the patches, consisting of added lines and deleted lines, which are denoted as C_{add} and C_{del} , respectively, as well as their corresponding fixing commit ID for the given CVE ID.

3.3.1 Critical Function Selection. VMUD obtains the files before and after change using the gathered commit ID. Then, it transforms the files into the Abstract Syntax Tree (AST). Using the added lines (C_{add}) and the deleted lines (C_{del}) for each file, VMUD locates the modified, added, and deleted functions, denoted as $\mathcal{F}^*$. To identify the critical functions, VMUD first generates a local call graph for the target project which is directly related to the vulnerabilities. Then, VMUD utilizes the PageRank algorithm [47], a widely recognized algorithm for assessing the significance of web pages in search engine rankings. This choice stems from the analogy between the structural characteristics of a call graph and the linkages among web pages, which are both represented as directed graphs. The algorithm is expert at identifying web pages that are most likely to be visited. Likewise, the task of identifying critical functions closely aligns with identifying the most frequently accessed functions, as they are more likely to be exploited by other functions. Therefore, we determine those functions as critical functions. Instead of processing an entire call graph, the algorithm operates on a local graph centered around the changed functions. Li et al. [28] suggest that vulnerabilities often involve multiple functions spanning an average depth of 2.8 invocation levels. Accordingly, VMUD focuses on callers and callees of $\mathcal{F}^*$ within a three-level invocation depth. It captures the local graph, comprising invocation relationships up to three levels of invocation depth, both upstream and downstream. The algorithm outputs a list of significance scores for each function in the local graph. We use a threshold variable th_{pr} to filter out the functions from $\mathcal{F}$ with low significance scores and keep the rest as critical functions, denoted as $\mathcal{F}'$.

3.3.2 Program Rephrasing. Previous studies [23, 49, 50, 54] do not consider the VCCs expressed in different syntactic forms yet equivalent in semantic meanings, leading to potential false negatives. To address this, we employ program rephrasing techniques to enrich the syntactic diversity of vulnerability signatures. Program rephrasing, a form of program transformation [43], reshapes code while preserving its semantic meaning. This approach allows us to detect more VCCs manifesting in diverse syntactic structures. We primarily focus on the equivalent rephrasing in the C/C++ language. Specifically, we refer to the C11 language standard documentation [36], which is a comprehensive manual illustrating the syntaxes and grammar of C/C++ in C11. We search the documentation containing the “equivalent” keyword and find 160 locations mentioning

```

1 ngx_int_t ngx_mail_read_command(ngx_mail_session_t *s,
2 ngx_connection_t *c)
3 {...
4     ngx_int_t rc;
5     if (rc == NGX_IMAP_NEXT || rc == NGX_MAIL_PARSE_INVALID_COMMAND){
6         if (rc == NGX_MAIL_PARSE_INVALID_COMMAND) {
7             s->errors++;
8             if (s->errors >= cscf->max_errors) {
9                 ngx_log_error(NGX_LOG_INFO, c->log, 0,
10                    "client sent too many invalid commands");
11                 s->quit = 1;
12             }
13             return rc;
14         }
15         if (rc == NGX_IMAP_NEXT)
16             return rc;

```

(a) Patch Code for CVE-2021-3618

```

1 // Retry operation to continue completion.
2 #define NGX_AGAIN -2
3 // Current operation completed; proceed to next step.
4 #define NGX_DONE -4
5 // Current operation successfully completed.
6 #define NGX_OK 0
7 ngx_int_t ngx_http_core_access_phase(ngx_http_request_t *r,
8 ngx_http_phase_handler_t *ph)
9 {
10     ngx_int_t rc;...
11     if (rc == NGX_AGAIN || rc == NGX_DONE) {
12         return NGX_OK;
13     }

```

(b) A False Alarm in Target Project

Figure 3: Examples of Program Rephrasing

the keyword. We look into the 160 locations and conclude 15 distinct types of semantic equivalence patterns. Among them, 8 types are strongly related to C/C++ library APIs and 3 types are not commonly used. As a result, we summarize the following program rephrasing rules that can help to enhance our signature representation:

- **Macros.** Existing works [23, 49, 50, 54] regard macro as either function calls, parameters or local variables. Given a statement s containing a macro, we rephrase it by expanding the macro at the statement. For example, the statement containing the macro $\text{MAX}(a, b)$ would be rephrased to $a > b ? a : b$.
- **Indirection Expression:** $\&a$ and $\&*a$ can be rephrased to a .
- **Logical NOT Expression:** $!E$ can be rephrased to $0 = E$ and $!!E$ can be rephrased to E .
- **Comparison Expression:** $a >= b$ can be rephrased to $a > b || a == b$.

We transform the source code of the critical functions $\mathcal{F}'$ into the Abstract Syntax Tree (AST) representation. Then, based on the above rephrasing rules, we transform it into the semantically equivalent form according to the rules on the AST and convert it back to source code. The rephrased functions are denoted as $\mathcal{F}'_{rep}$.

Example 3.1. Figure 3 illustrates an instance of program rephrasing. In Figure 3(a), we depict a patch addressing the vulnerability in `ngx_mail_read_command` identified in CVE-2021-3618. Figure 3(a) presented a false alarm reported by VUDDY [23], MVP [54], MOVER [50] and V1SCAN [49]. The vulnerable code snippet comprises a variable declaration statement (Line 4), an if statement (Line 5), and a return statement (Line 16). Notably, the if statement incorporates two comparison expressions, a frequently encountered construct, each involving a macro. Existing tools [23, 49, 50, 54] abstract these macros into local variables, resulting in false alarms. In contrast, our tool (VMUD) replaces the macros with their corresponding expressions. For instance, the macro `NGX_DONE` gets transformed into the number literal `-4`.

3.3.3 Signature Generation. VMUD generate the original signatures X_{org} for functions $\mathcal{F}'$ and rephrased signatures X_{rep} for $\mathcal{F}'_{rep}$. It consists of Program Slicing, Abstractions and Normalization, and Semantic Enhanced Signature Generation.

Program Slicing. VMUD generates the Program Dependence Graph (PDG) for the critical functions $\mathcal{F}$. It utilizes the deleted lines C_{del} and added lines C_{add} and conducts forward and backward slicing based on the PDG.

Abstraction and Normalization. The reused code may undergo parameter renaming, variable renaming, and code style changes. We abstract and normalize the original functions f_{org} as well as rephrased functions f_{rep} . Specifically, for each statement in Syn , Sem , and See , we apply abstraction and normalization. We abstract function parameters, type declaration, local variables, function calls, and string literals in the code into “FPARAM”, “DTYPE”, “LVAR”, “FUNCCALL”, and “STRING”, respectively [23]. In string literals, we retain format specifiers which have semantic meaning for specific placeholders. For example, we abstract “`protos=%s`” to “`%s`”. We also remove comments, braces, spaces, and tab characters from the program [54].

Semantic Enhanced Signature Generation. Syn is a list comprising of the code statements s in the function, denoted as $\langle s \rangle$. Each statement is a 2-tuple $\langle l, t \rangle$, where l is the line number and t is the text of the statement. Sem comprises the semantic relationship of the statements, denoted as a 3-tuple $\langle s_1, s_2, type \rangle$. The $type$ can either be *control* or *data* dependency. Existing work [54] compares the similarity between the signature and target function based on Syn and Sem . Differently, VMUD creates Semantic Enhanced (SeE) signature See to obtain the Contextual Equivalent Statements Map (see Section 3.4.3).

To this end, VMUD extract the semantic enhanced sets See for vulnerability signatures V and patched signatures P . Specifically, for each statement $s \in Syn$, we iterate the entries in Sem . For each entry $\langle s_1, s_2, type \rangle$, if s either equals s_1 or s_2 , we put $\langle s_1, s_2, type \rangle$ into See (see Eq. 1). X_{src} can either be X_{org} or X_{rep} .

$$See = \{ \langle l, \langle s_i, s_j, type \rangle \rangle \mid \forall s.l \in X_{src}.Syn, \langle s_i, s_j, type \rangle \in X_{src}.Sem, \text{ if } s.l \in \{s_i.l, s_j.l\} \} \quad (1)$$

To accelerate the comparison, we leverage the MD5 hash to compute the hash value for each statement s after abstraction and normalization. Finally, we obtain $\langle Syn, Sem, See \rangle$ for vulnerable function V and patched functions P regarding the original signature X_{org} and the rephrased signature X_{rep} .

3.4 VM Detection

VMUD detects the VM in the target project using X_{org} and X_{rep} . It consists of Preprocessing, Signature Generation, and Signature Matching. The Signature Generation includes the Program Rephrasing, Abstraction, and Normalization (see Section. 3.3.2 and 3.3.3).

3.4.1 Preprocessing. For efficient detection, we conduct preprocessing on the target project. Similar to search space reduction in MOVER [50] and code classification of reused and vulnerable code in V1SCAN [49], the purpose of the preprocessing is to reduce the search space, focus on relevant clones, and filter out the functions that are less likely to be code clones. We leverage SAGA [25], an

effective clone detection tool. We conduct snippet-level clone detection with the minimum token of snippet set as 20 token to ensure high recall rates. Specifically, the length of the snippet is determined by the number of tokens. SAGA finds a cloning snippet if there is one snippet equal in functions from $\mathcal{F}_{src}$ and $\mathcal{F}_{tgt}$. Finally, we obtain the functions that may contain the VCCs, denoted as $\mathcal{F}_{tgt}$.

3.4.2 Signature Generation. Given input as candidate functions from target project $\mathcal{F}_{tgt}$, we generate a signature for each function and its semantically equivalent functions, denoted as X_{tgt_org} and X_{tgt_rep} . For each X_{tgt_org} and X_{tgt_rep} , we generate a syntactic signature *Syn*, semantic signature *Sem*, and semantic enhanced signature *See* following same signature generation in Section 3.3.3.

3.4.3 Signature Matching. The signature matching process comprises two phases (see Figure 2). In **Phase I**, the signatures X_{tgt_org} of target project functions are matched against the original vulnerability signatures X_{org} . In **Phase II**, the Rephrased target signatures X_{tgt_rep} are compared with rephrased vulnerability signatures X_{rep} . In each phase, VMUD evaluates syntactic similarity (X_{tgt_Syn} vs. $X_{src_Syn}^V$ and $X_{src_Syn}^P$) as well as semantic similarity (X_{tgt_Sem} vs. $X_{src_Sem}^V$ and $X_{src_Sem}^P$), where X_{src} can be either X_{org} or X_{rep} and X_{tgt} can be either X_{tgt_org} or X_{tgt_rep} . We refine the similarity assessment using X_{tgt_See} , $X_{src_See}^V$, and $X_{src_See}^P$ (see Eq. 6). We incorporate the Contextual Equivalent Statement Map (CESM) to ensure that only contextually and semantically equivalent statements contribute to the similarity metric.

We run Phase I and Phase II under four conditions. The rationale is rooted in the principle that a signature of a VCC should be similar to the vulnerable function signature while different from the patched function signature. False negatives occur when the target function signature is different from the vulnerable function's signature. Therefore, Phase II is executed under the condition that the target function signature X_{tgt} aligns with the vulnerable function signature X_{org}^V while differing from the patched function signature X_{org}^P . We describe the conditions using the following rules:

- **Run Phase I:** If $\text{sim}(X_{tgt_org}, X_{org}^V)$ is true and $\text{sim}(X_{tgt_org}, X_{org}^P)$ is false, report a positive vulnerability alarm for the target function and skip **Phase II**.
- **Run Phase I:** If $\text{sim}(X_{tgt_org}, X_{org}^V)$ is true and $\text{sim}(X_{tgt_org}, X_{org}^P)$ is true, report a negative result for the target function and skip **Phase II**.
- **Run Phase I:** If $\text{sim}(X_{tgt_org}, X_{org}^V)$ is false, then **Run Phase II:** If $\text{sim}(X_{tgt_rep}, X_{rep}^V)$ is true and $\text{sim}(X_{tgt_rep}, X_{rep}^P)$ is false, report a positive vulnerability alarm for the target function.
- **Run Phase I:** If $\text{sim}(X_{tgt_org}, X_{org}^V)$ is false, then **Run Phase II:** If $\text{sim}(X_{tgt_rep}, X_{rep}^V)$ is true and $\text{sim}(X_{tgt_rep}, X_{rep}^P)$ is true, report a negative result for the target function.

The similarity function sim is defined in Eq. 2. Literally, it means that the signature of the target function should be syntactically (i.e., $\text{syn}()$) and semantically (i.e., $\text{sem}()$) similar to the source function (i.e., V or P). Meanwhile, the deleted statements at the source function should exist in the target function (i.e., $\text{exist}()$).

The similarity function sim is formalized in Eq. 2. It denotes that the target function's signature X_{tgt} should have both syntactic and semantic similarity to the vulnerability function X_{src} . Additionally,

it also requires that any statements deleted in the source function should be present in the target function.

$$\begin{aligned} \text{sim}(X_{tgt}, X_{src}) = & \text{syn}(X_{tgt}, X_{src}) \\ & \& \text{sem}(X_{tgt}, X_{src}) \\ & \& \text{exist}(X_{tgt}) \end{aligned} \quad (2)$$

The $\text{syn}()$ function measures the syntactic similarity between the syntax signatures of X_{tgt_Syn} and X_{src_Syn} . Similarly, the $\text{sem}()$ function evaluates their semantic similarity between X_{tgt_Sem} and X_{src_Sem} . The $\text{exist}()$ function verifies the presence of deleted statements S_{del} from X_{src} in the target project X_{tgt} . These functions are defined as follows:

$$\text{syn}(X_{tgt}, X_{src}) = 1 \text{ if } \frac{|\text{cesm.keys}|}{|X_{src_Syn}|} > th_{syn}, \text{ and 0 otherwise} \quad (3)$$

$$\text{sem}(X_{tgt}, X_{src}) = 1 \text{ if } \frac{|\text{Sem}_{pft} \cap \text{Sem}_{pfv}|}{|\text{Sem}_{pfv}|} > th_{sem} \text{ and 0 otherwise} \quad (4)$$

$$\text{exist}(X_{tgt}) = 1 \text{ if } \forall s \in S_{del}, s \in X_{tgt_Syn} \text{ and 0 otherwise} \quad (5)$$

The computation of $\text{syn}()$ involves generating the Contextual Equivalent Statements Map (CESM) through the $\text{CESM}()$ function, as defined in Eq. 6. This function produces a CESM output denoted as a tuple $\langle s_j, \langle s_i \rangle \rangle$, where s_j represents a statement in X_{src} and s_i denotes a statement in X_{tgt} . It implies that for a given statement s_j in X_{src} , there are several statements s_i in X_{tgt} that are contextually equivalent to s_j . Unlike traditional equality comparisons (i.e., using $=$ or $==$), the $\text{CESM}()$ function determines the equivalence between statements based on their contextual equivalence.

For any given $s_i \in X_{tgt}$ and $s_j \in X_{src}$, VMUD first obtains semantic relationships. i.e., $X_{tgt_See}[s_i.l]$ for s_i and $X_{src_See}[s_j.l]$ for s_j . It then computes the intersection of these sets and divides it by $X_{src_See}[s_j.l]$. If the intersection ratio exceeds the threshold th_{ce} , VMUD considers the two sets as contextually similar, thus concluding that the s_j and s_i are contextually equivalent. cesm.keys denotes the keys of the cesm output (see cesm.keys in Eq. 3).

$$\begin{aligned} \text{cesm} = \text{CESM}(X_{tgt}, X_{src}) = & \{ \langle s_j, \langle s_i \rangle \rangle \mid \forall s_i \in X_{tgt}, s_j \in X_{src}, \\ & \frac{|X_{tgt_See}[s_i.l] \cap X_{src_See}[s_j.l]|}{|X_{src_See}[s_j.l]|} > th_{ce} \} \end{aligned} \quad (6)$$

In computing $\text{sem}()$, we utilize partial functions, which are a subset of statements from X_{src} or X_{tgt} . Specifically, with the contextual equivalent statements identified from the target and source functions (see Eq. 6), our goal is to identify the subset of statements within the target function that exhibits the strongest semantic relationship with the source function. To this end, it involves the search problem where for each entry $\langle s_j, \langle s_i \rangle \rangle$, we need to find an optimal statement s_i from $\langle s_i \rangle$ for each s_j , such that the semantic relationship between X_{tgt} and X_{src} is maximized. We use the $\text{argmax}()$ function denoted in Eq. 7.

$$\text{argmax}(\frac{|\text{Sem}_{pft} \cap \text{Sem}_{pfv}|}{|\text{Sem}_{pfv}|}) \quad (7)$$

Table 3: Number and Percentage of Functions in Collected VM

Function (#)	VM (#)	Percentage of VM
2	333	41.11%
3	146	18.02%
4	97	11.97%
5	56	6.91%
6	42	5.19%
7	32	3.95%
8	20	2.47%
9	10	1.23%
10	14	1.73%
>10	60	7.41%
Total	810	100.00%

Sem_{pfs} denotes the semantic signature of the partial function of the source function (see Eq. 8). Syn_{pft} and Sem_{pft} denote the syntactic and semantic of the partial function of the target function.

$$\begin{aligned}
Sem_{pfs} &= \{ \langle s_1, s_2, type \rangle \mid s_1 \text{ or } s_2 \text{ in } cesm.keys \} \\
Syn_{pft} &= \{ s_i \mid \exists s_i \in cesm[s_j], \forall s_j \in cesm.keys \} \\
Sem_{pft} &= \{ tup \mid tup.s_1 \text{ or } tup.s_2 \in Syn_{pft}, \forall tup \in Sem_{tgt} \}
\end{aligned} \quad (8)$$

4 IMPLEMENTATION

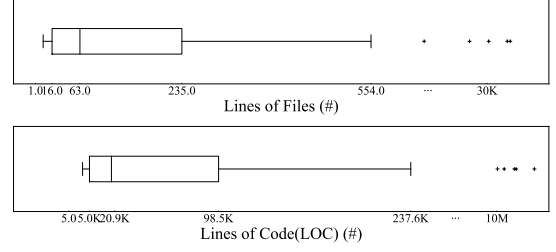
We implemented VMUD using 1,900 lines of Python code. To generate the call graph for the project before vulnerability fixing commits, we utilized Doxygen [9]. Due to Doxygen’s scalability limit in analyzing large amounts of files, we prioritized neighboring source code files based on their path relativity to the changed files in the vulnerability fixing commit. Then, we limited Doxygen’s analysis input to a maximum of 5,000 files. The threshold score of PageRank [47], denoted as th_{pr} , was set at 0.018 based on our sensitivity evaluation (see Section 5.5). We extracted patch statements using the *git show* command and leveraged Joern [39] to generate a comprehensive code property graph, including Abstract Syntax Trees, Control Flow Graphs, and Program Dependence Graphs. We employed the GCC compiler [12] for program rephrasing on macros.

For collecting CVEs, we accessed the official API [35] in 2023, which contains the data in the recent decade. We obtained corresponding fixing commits on 1,782 CVEs, filtering out 972 whose fixing functions are less than 2. As a result, we obtained 810 VM with 4,551 fixing functions. The statistics are presented in Table 3.

5 EVALUATION

We evaluate VMUD based on the following research questions.

- **RQ1. Effectiveness Evaluation.** How is the effectiveness of VMUD compared to the state-of-the-art tools?
- **RQ2. Robustness Evaluation.** How is the robustness VMUD in detecting VM?
- **RQ3. Ablation Study.** How does each component contribute to VMUD’s effectiveness?
- **RQ4. Threshold Sensitivity.** How do the thresholds contribute to the effectiveness of VMUD?

**Figure 4: Number of Files and Lines of Code (LOC) on our Evaluated Dataset**

- **RQ5. Performance Evaluation.** How is the efficiency of VMUD?

5.1 Setup

Target Project Selection. We established three criteria for project selection. Firstly, projects should be written in C/C++. Secondly, they should be popular in popularity and span diverse domains. Thirdly, they should be actively maintained. Subsequently, we collected 972 projects from GitHub by querying the Top 1000 popular projects ranked by their stars using GitHub API [15]. These projects encompass areas such as databases, operating systems, image processing, reverse development, etc. Figure 4 depicts the number of files and lines of code (LOC) across our selected projects using a box plot.

Comparison Tools. We chose four clone-based tools, namely Vuddy [23], MVP [54], Moverly [50], and V1Scan [49], recognized as state-of-the-art in VCC detection. Additionally, we included two learning-based tools, SySeVR [30] and DeepDFA [40], to assess their effectiveness compared with VMUD. We used the default configurations outlined in the original papers for all selected tools.

Ground Truth Construction. To identify a comprehensive set of VM, we first executed all tools on the target projects. Subsequently, we conducted manual validation of all positive results by the authors to confirm the presence of VM. This process involved two authors independently classifying each positive result, i.e., whether it is a true positive or a false positive. Any discrepancies were resolved through discussion, as well as a third author consulted if necessary. We use Cohen’s Kappa coefficient to measure agreement, and it reached 0.951. The three authors have over three years of experience in C/C++ development. This process led to the creation of a ground truth dataset containing 329 confirmed true VM.

Evaluation Configuration. We conducted our evaluations of VMUD and the comparison tools on a machine equipped with a 2.10GHz Intel Xeon processor and 256GB of RAM. We employed true positives (TP), false positives (FP), false negatives (FN), precision, recall, and F1-Score [48] to measure the tools’ effectiveness and accuracy w.r.t the distinct vulnerability with multiple vulnerable clones. Specifically, we define one TP when the evaluated tool reports a vulnerability warning due to a match with at least one of its fixing functions of a VM. We record one FP when the tool reports false vulnerability warnings based on matching its fixing functions to one VM. We denote one FN when the project contains a VM, but the tool fails to identify any of its matching functions as vulnerable.

Table 4: Effectiveness and Accuracy Results of VMUD and Clone-Based Tools (* matching-one-function-in-all approach and & denotes the matching-all-functions approach)

Dataset	Tool	TP (#)	FP (#)	FN (#)	Precision	Recall	F1-Score
Complete VM Dataset	VUDDY*	168	25	161	0.87	0.51	0.64
	MVP*	226	163	103	0.58	0.69	0.63
	V1SCAN*	113	47	216	0.71	0.34	0.46
	VMUD *	275	51	54	0.84	0.84	0.84
	VUDDY&	47	2	282	0.96	0.14	0.25
	MVP&	48	3	281	0.94	0.15	0.25
	V1SCAN&	32	1	297	0.97	0.10	0.18
	VMUD &	59	2	270	0.97	0.18	0.30
	MOVERY*	82	56	27	0.59	0.75	0.66
	VMUD *	90	10	19	0.90	0.83	0.86
Diminished VM Dataset	MOVERY&	17	3	94	0.85	0.16	0.26
	VMUD &	29	1	80	0.97	0.27	0.42

RQ Setup. To study **RQ1**, we ran all tools on our ground truth dataset (see Section 4). However, As MOVERY does not provide open-source code for signature generation, we intersected our collected CVEs with their dataset to ensure a fair evaluation, which resulted in a Diminished VM Dataset of 144 VM signatures. Therefore, MOVERY is not included in the comparisons shown in Figure 5 and Figure 6. Furthermore, we assessed each clone-based tool under two settings to explore the relationship between fixing functions. A superscript * denotes a tool reporting a VM upon matching any fixing function signature (*i.e.*, the matching-one-function-in-all approach), while a superscript & denotes reporting a VM only when matching all fixing function signatures (*i.e.*, the matching-all-functions approach). Additionally, we compared two learning-based tools using precision and recall w.r.t vulnerable functions. To study **RQ2**, we observed VMUD’s robustness by evaluating its accuracy across VM with varying numbers of fixing functions. To investigate **RQ3**, an ablation study was conducted to assess VMUD’s effectiveness in critical function selection, Program rephrasing, and contextual equivalent statement mapping. **RQ4** involved evaluating the sensitivity of thresholds in VMUD, while **RQ5** focused on its overall performance.

5.2 Effectiveness Evaluation (RQ1)

We evaluate the accuracy and effectiveness of VMUD with state-of-the-art tools using our ground truth dataset. Additionally, we compare VMUD against learning-based approaches. Subsequently, we analyze the inaccuracies.

5.2.1 Accuracy Results. Table 4 presents the effectiveness and accuracy results of VMUD and the clone-based tools on our ground truth dataset. Overall, VMUD achieves a precision, recall, and F1-Score of 0.84. In comparison, VUDDY* achieves the highest precision of 0.87 but only reaches a recall of 0.51. MOVERY* reaches the highest recall of 0.75 and the highest F1-Score of 0.66. Notably, VMUD outperforms MOVERY with an F1-Score increase of 17.6%. Additionally, using the matching-all-functions approach (denoted by &) significantly boosts precision while lowering recall, which may be impractical in most scenarios.

Accuracy Per Project We calculated the accuracies of each tool’s results on every project containing VM. Figure 5 displays

Table 5: Target Projects with the Confirmed or Fixed VM

Target Project	TP (#)	FP (#)	FN (#)	Precision	Recall	Confirmed (#)	Fixed (#)
seemoo-lab/nexmon	4	0	2	1.00	0.67	4	0
panda-re/panda	4	0	1	1.00	0.80	4	0
rizinorg/rizin	6	0	1	1.00	0.86	3	3
Cisco-Talos/pyrebox	3	0	1	1.00	0.75	3	0
freebsd/freebsd-src	4	1	0	0.80	1.00	2	2
civetweb/civetweb	3	0	0	1.00	1.00	2	2
nodemcu/nodemcu-firmware	2	0	0	1.00	1.00	2	2
radareorg/radare2	2	0	0	1.00	1.00	2	2
gpac/gpac	2	0	0	1.00	1.00	2	2
catboost/catboost	2	0	0	1.00	1.00	2	0
TelegramMessenger/Telegram-iOS	4	0	0	1.00	1.00	2	2
ravynsoft/ravynos	4	0	0	1.00	1.00	1	1
mridgers/clink	1	0	0	1.00	1.00	1	1
eduard-permyakov/permafrost-engine	1	0	0	1.00	1.00	1	1
ntop/PF_RING	1	0	0	1.00	1.00	1	0
wireshark/wireshark	1	0	0	1.00	1.00	1	1
libretro/RetroArch	1	1	0	0.50	1.00	1	1
ApsaraDB/PolarDB-for-PostgreSQL	2	0	0	1.00	1.00	1	1
momotech/MLN	1	0	0	1.00	1.00	1	1
slact/nchan	1	0	0	1.00	1.00	1	1
Proxmark/proxmark3	1	0	0	1.00	1.00	1	1
sdlpal/sdlpal	1	0	0	1.00	1.00	1	1
openzfs/zfs	1	0	0	1.00	1.00	1	1
ImageMagick/ImageMagick	1	1	0	0.50	1.00	1	1
alibaba/tengine	1	1	0	0.50	1.00	1	1
Total	54	4	5	0.93	0.92	42	28

the distribution of projects based on precision, recall, and F1-Score thresholds. The metrics with NaNs (*i.e.*, either TP+FP=0 or TP+FN=0) are not included. Notably, VMUD detects more projects with precision ranging from 0.2 to 0.9 compared to other tools. It reports 2 fewer projects than VUDDY and V1SCAN with a precision of 1.0. Additionally, VMUD demonstrates significant advantage across the evaluated projects in terms of recall and F1-Score, surpassing other tools in any recall or F1-Score values.

5.2.2 Effectiveness Results. VMUD identified 275 true VM across 84 projects. Among these, developers confirmed 42 VM from 25 projects, 28 VM from 20 projects have been resolved, and 14 VM from 5 projects await resolution in the next version. In contrast, VUDDY detected 168 true VM across 63 projects, MVP detected 226 true VM across 74 projects, MOVERY detected 82 true VM across 36 projects, and V1SCAN detected 113 true VM across 43 projects. Table 5 details the projects with confirmed or resolved VM of VMUD. Additionally, 5 VM are disclosed and assigned with 5 CVE identifiers.

Effectiveness Per Project Figure 6 illustrates the number of true positives (TPs), false positives (FPs), and false negatives (FNs) detected by VMUD and clone-based tools across projects with VM. To equally compare effectiveness per project, we choose projects that contain at least one TP, FP, or FN in the results across all four tools. The project identifier used in the graph corresponds to the project names listed in the appendix table (see Table 10). We excluded MOVERY due to the diminished VM dataset. Specifically, VMUD identifies more correct VM (*i.e.* identifies more true positives) than any other tool in 29 projects, accounting for 87.9% of the projects with VM. Moreover, VMUD reports fewer false positives than other tools in 19 projects and fewer false negatives in 14

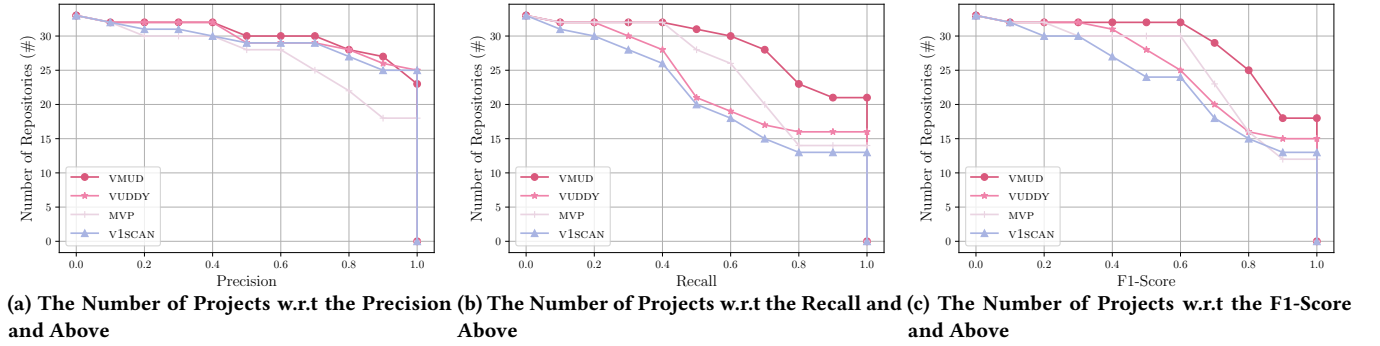


Figure 5: The Numbers of Projects w.r.t the Precision, Recall, and F1-Score and above

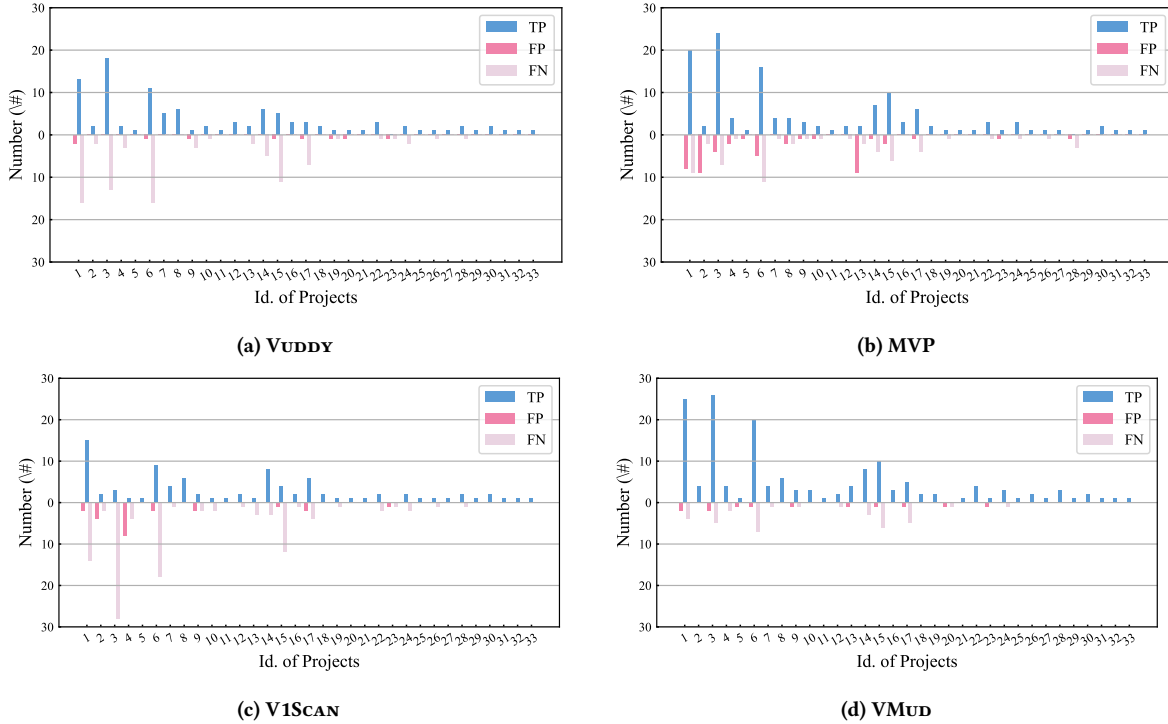


Figure 6: The Number of True Positives, False Positives, and False Negatives Detected by VMUD and Clone-based Tools

projects, representing 57.6% and 42.4% of the projects with VM, respectively.

5.2.3 Comparison with Learning-based Approaches. We conducted a comparative analysis of VMUD's accuracy with DEEPDFA [40] and SySeVR [30], two leading learning-based approaches. We directly utilized the trained models of DeepDFA and SySeVR on their respective datasets and used our ground truth as the testing dataset. Table 6 presents the results of these learning-based approaches. Notably, the precision and recall metrics in their context are measured concerning reporting vulnerable functions, which differs from our definition. Therefore, we present VMUD's precision and recall based on their definition to facilitate comparison, denoted by a superscript F . Consequently, SySeVR achieves a precision of 0.58 and a recall of 0.05, while DEEPDFA achieves a precision of 0.68 and a recall of 0.81. In comparison, VMUD achieves a precision of 0.83

Table 6: Accuracy Results of Learning-Based Approaches

Tool	Precision F	Recall F	F1-Score F
SySeVR	0.58	0.05	0.10
DEEPDFA	0.68	0.81	0.74
VMUD	0.83	0.79	0.81

recall of 0.79, and an F1-Score of 0.81, marking a 7.5% improvement over DEEPDFA concerning F1-Score.

5.2.4 Inaccuracy Analysis. We conduct analysis and summarize reasons for the inaccuracies of VMUD.

False Positives Analysis VMUD generated 51 false positives, which are primarily caused by two key factors. Firstly, some statements have limited semantic relationships, such as function invocations lacking input parameters (e.g., `invoke()`) or return statements with literals (e.g., `return 0`). In these cases, matching semantic similarity is less effective than matching syntactic similarity.

Table 7: VMUD's Robustness in Detecting VM with Multiple Numbers of Fixing Functions

Fixing Function (#)	TP (#)	FP (#)	FN (#)	Precision	Recall	F1-Score
2	134	15	18	0.90	0.88	0.89
3	33	11	5	0.75	0.87	0.80
4	35	8	10	0.81	0.78	0.80
5	19	4	3	0.83	0.86	0.84
6	9	0	3	1.00	0.75	0.86
7	18	4	1	0.82	0.95	0.88
8	5	3	0	0.63	1.00	0.77
9	1	0	3	1.00	0.25	0.40
10	3	1	1	0.75	0.75	0.75
>10	18	4	10	0.82	0.64	0.72

Consequently, VMUD relies more heavily on syntactic similarity despite their contextual differences, resulting in 29 false positives. Secondly, for vulnerable functions that have already been patched, downstream developers proceeded with subsequent customizations, which altered the patched statements. Typically, the patched statements are concise, comprising a small of statements. Consequently, a single alternation to these patched statements can cause a significant drop in similarity with the original patched statements, leading to 22 false positives.

False Negatives Analysis. VMUD encountered 54 false negatives, primarily due to three reasons. First, in our preprocessing step (see Section 3.4.1), we utilized a fragment-level clone detection tool SAGA[25]. While this preprocessing helped eliminate numerous unnecessary detections while preserving VCCs in most cases, it did result in 4 false negatives. Secondly, VMUD labels a vulnerable function as safe if its signature matches both the vulnerable and patched functions from the VM. However, certain patches are trivial, leading to a significant resemblance in the signatures of vulnerable and patched functions. This high resemblance causes VMUD to mistakenly categorize the vulnerable function as the patched, ultimately classifying the vulnerable function as safe, which results in 22 false negatives. Thirdly, similar to false positives caused by downstream customizations, there are instances where vulnerable functions have been patched, but downstream customizations have affected the vulnerable statements. It causes the functions to exhibit lower similarity to the vulnerable function signature, resulting in 28 false negatives.

5.3 Robustness Evaluation (RQ2)

To assess VMUD's robustness, we examine its accuracy across VM with varying numbers of fixing functions. Table 7 depicts VMUD's precision and recall concerning VM with fixing function counts ranging from 2 to 10, and above 10. We note that VMUD achieves maximum precision and recall of 1.00 for VM and a minimum precision of 0.63 and recall of 0.25 for VM with different fixing function numbers. Nevertheless, certain categories may have a limited number of VM, which could impact the accuracy assessment. Overall, VMUD maintains an F1-Score over 0.8 across fixing function counts from 2 to 7, indicating its robustness in detecting VM, particularly those with multiple fixing functions.

5.4 Ablation Study (RQ3)

To assess the impact of key components in VMUD, we conducted ablation study where these components were individually removed,

Table 8: Results of the Ablation Study (CFS denotes the Critical Function Selection, PR denotes the Program Rephrasing, CESM denotes the Contextual Equivalent Statement Matching)

	TP (#)	FP (#)	FN (#)	Precision	Recall	Δ Precision	Δ Recall
w/o CFS	305	216	24	0.59	0.93	-0.25	0.09
w/o PR	249	46	80	0.84	0.75	-0.00	-0.09
w/o CESM	270	81	59	0.77	0.82	-0.07	-0.02

and then evaluated the effectiveness of VMUD. Table 8 summarizes the effectiveness results of the ablated versions. Notably, the Signature Matching in Phase II relies on the rephrased signature generated by Program Rephrasing. Therefore, in the ablated version VMUD w/o PR, both the Signature Matching in Phase II and Program Rephrasing are jointly removed. The ablated versions of VMUD show decreases in precision and recall compared to the original VMUD. Specifically, VMUD without Critical Function Selection experiences a precision drop of 0.25, with a 0.09 increase in recall. The increase in recall is due to the inclusion of recovered functions thus helping in VM detection. In VMUD without Program Rephrasing and Signature Matching in Phase II, the precision remains unchanged, and recall drops by 0.09. In VMUD without Contextually Equivalent Statements Map (CESM), the precision drops by 0.07, with a 0.02 decrease in recall. The results indicate that the three components of VMUD all contribute to the effectiveness of VMUD.

5.5 Threshold Sensitivity (RQ4)

We conducted a sensitivity analysis to assess the impact of various thresholds (th_{pr} , th_{syn}^V , th_{sem}^V , th_{syn}^P , th_{sem}^P , th_{ce}) on VMUD's performance. Figure 7 illustrates how each threshold influences the recall, precision, and F1-Score of VMUD. The optimal performance for VMUD is achieved when the thresholds are set as 0.018, 0.7, 0.6, 0.3, 0.4, 0.6, respectively.

5.6 Performance Evaluation (RQ5)

We evaluated the time cost of VMUD and several clone-based tools across a set of 972 target projects, specifically focusing on the time taken to detect VM. Figure 8 displays a box plot illustrating the time spent on VM detection in each project. On average, VMUD takes a median of 227 seconds to detect VM in a project, which is 173, 45, 142, 178 seconds longer than VUDDY, MVP, MOVEVER, and V1SCAN. The time cost of VMUD primarily stems from call graph generation and semantic equivalent statement matching, which are computationally intensive. However, we consider this additional time cost acceptable given VMUD's effectiveness in detecting VM in real-world projects compared to other tools.

5.7 Discussion

Threats. Our evaluated results are affected by three potential threats. First, concerning our ground truth dataset, it's constructed based on the manual analysis of detected vulnerabilities from VMUD and other clone-based tools. This approach may introduce bias as not all vulnerabilities may be detected by these tools, impacting the accuracy of our ground truth dataset. However, we've utilized existing tools comprehensively to cover a broad range of vulnerabilities. Second, in our effectiveness evaluation, we compared MOVEVER on

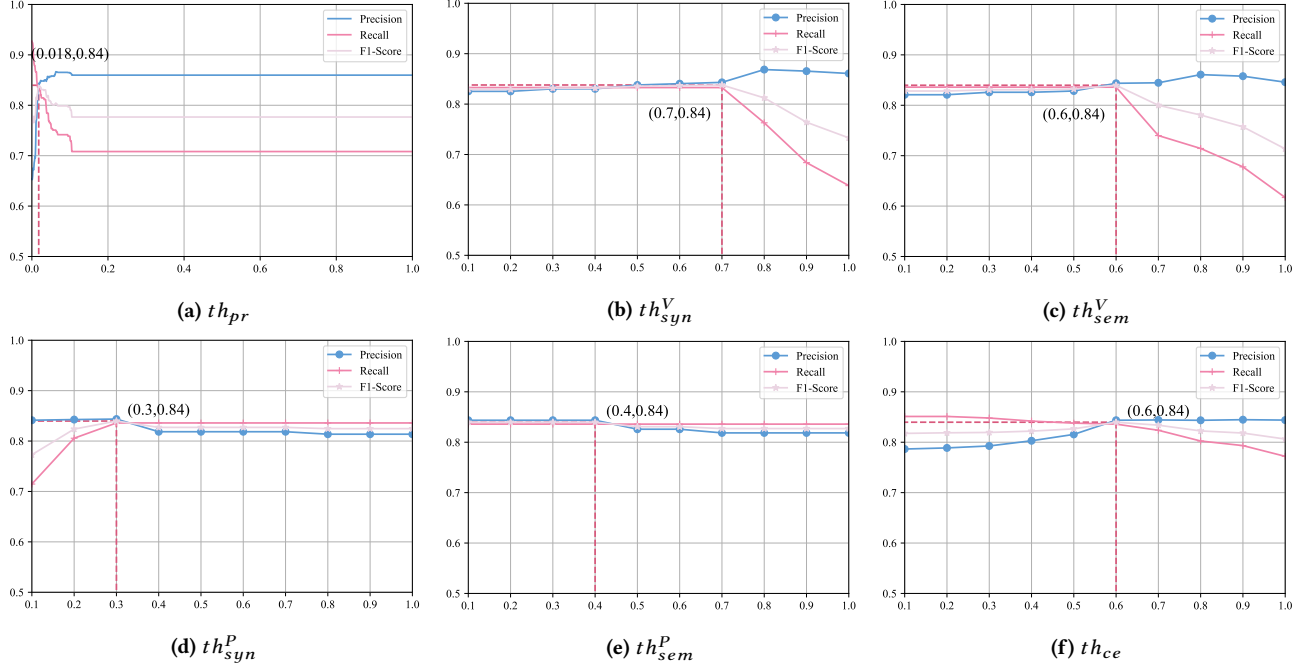


Figure 7: Results on Threshold Sensitivity

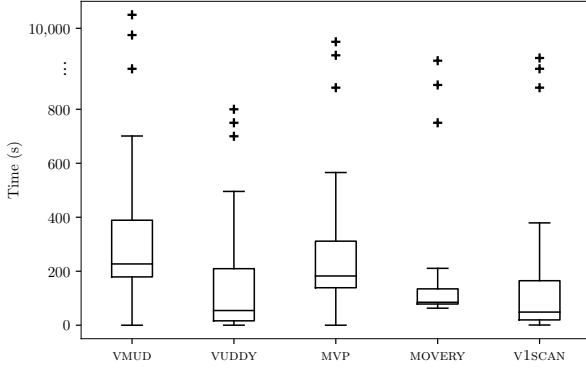


Figure 8: Performance

a smaller scale, which may not provide a complete comparison of precision and recall between MOVERY and other tools. Nonetheless, our evaluation of the small-scale dataset demonstrates VMUD's advantages over MOVERY fairly. Third, in our robustness evaluation, the distribution of VM concerning the number of fixing functions is uneven, with some categories having a small number of VM. This may not fully reflect VMUD's effectiveness in those categories. However, VMUD has consistently shown strong performance across most categories, highlighting its robustness.

Limitations. Our approach has several limitations. First, VMUD relies on several tools, limited by their accuracy and performance. For instance, Doxygen may generate incomplete call graphs, and Joern may produce incomplete CPGs, impacting critical function selection, signature generation, and signature matching. To address this, VMUD is designed to incorporate with other state-of-the-art tools to enhance its accuracy and performance. Second, VMUD

is designed for detecting vulnerabilities with multiple functions. We plan to collect and generate signatures for vulnerabilities with a single function, and adopt program rephrasing and contextual equivalent statement map to observe the effectiveness of VMUD. Third, VMUD does not maintain the knowledge of code clone evolution or customizations in the target project, potentially resulting in false negatives or false positives if significant modifications occur. However, we implement semantic-centered statement matching to ensure consistent semantics even amid code changes. We also plan to obtain evolutionary knowledge on code clones to enhance the effectiveness of VMUD.

6 RELATED WORK

We review previous work related to vulnerability detection, focusing on three aspects: vulnerability detection within a function, vulnerability detection beyond a function, and vulnerability abstraction and normalization.

Vulnerability Detection within a Function. The function serves as the fundamental unit for vulnerability detection, as it is the most common granularity level. Existing research usually transforms the task of detecting vulnerabilities across an entire program into individual tasks within each function. Many approaches employ clone detection techniques to identify vulnerable code clones (VCCs) within each function. For instance, Kim et al. [23] introduced VUDDY, which identifies VCCs by matching their clone signatures. Xiao et al. proposed MVP [54], leveraging program slicing to extract partial functions and match patching clone signatures, thus reducing false positives in VCC detection. Woo et al. [50] used the knowledge of the oldest vulnerable function to overcome the syntax diversity of vulnerable code. They further proposed V1Scan [49] to filter out unused vulnerable functions, thus reducing false alarms.

Additionally, learning-based approaches have been explored to classify vulnerable functions [6, 8, 38, 44, 58]. For example, Russell et al. [38] utilized CNN and DNN with Word2Vec embeddings for function-level code, while Zhou et al. introduced Devign [58] that encodes various function representations (*i.e.*, AST, CFG, DFG, NCS) into graph neural networks. Cui et al. [8] proposes a weighted feature graph (WFG) to compare functions in graph representation. Similarly, Wang et al. proposed GraphSPD [44] for multi-attributed graph convolution in vulnerability detection. However, for a vulnerability with multiple fixing functions, the vulnerability characteristics carried in each fixing function can vary depending on its surrounding context. Assigning equal importance to all fixing functions for a vulnerability may lead to over-representation, which brings false alarms. To this end, our approach applies critical function selection (see Section 3.3.1) to choose a subset of the fixing functions that are more significant, thus reducing false alarms.

Vulnerability Detection beyond a Function. Several studies have introduced methods for detecting vulnerabilities that go beyond the scope of individual functions. Software composition analysis (SCA) is instrumental in identifying vulnerabilities [19, 52, 53, 55]. By maintaining a database of vulnerable third-party libraries, SCA flags corresponding vulnerabilities based on instances of reused vulnerable libraries. However, it suffers from limitations in both precision and recall. This is because SCA primarily aims to characterize third-party libraries, which have a coarse-grained granularity compared to specific vulnerability characteristics.

Moreover, many approaches represent vulnerabilities at the file level or take the file-level program to analyze [21, 27, 33, 46]. Wi et al. [46] constructs a Control Flow Graph (CPG) for a PHP application and performs subgraph isomorphism analysis on it with a target PHP snippet to detect vulnerabilities across functions. Similarly, Khodayari et al. [21] create a Hybrid Property Graph (HPG) for each JavaScript file and identify CSRF vulnerabilities using declarative traversals. Mirsky et al. [33] customize an LLVM compiler toolchain to generate enriched Program Dependency Graphs (PDGs) for input files and apply program slicing and graph neural networks to identify vulnerabilities and their CWE types. While these methods are effective at detecting vulnerabilities within a single file, they may overlook vulnerabilities that span multiple files. Inter-procedural vulnerability detection methods, on the other hand, can identify vulnerabilities where the statements to be patched and the statements triggering the vulnerability belong to different functions [28]. These methods utilize inter-procedural program slicing [30, 31] or employ inter-procedural analyses to detect various vulnerability types [5, 10, 14, 20, 26, 32]. However, they require prior knowledge of specific CWE types and sophisticated inter-procedural analysis. In contrast, our approach can detect vulnerabilities with multiple functions without prior knowledge of CWE types.

Vulnerability Abstraction and Normalization. Vulnerability abstraction and normalization aim to create an intermediate representation for comparing original vulnerabilities with their instances in the target program. Learning-based approaches perform implicit abstraction and normalization in high-dimensional spaces, which makes it hard to interpret the manifestation of abstracted and normalized vulnerabilities. In contrast, clone-based methods explicitly encode the abstraction and normalization. For instance,

Li et al. [29] address missing code parts (*e.g.*, missing `}`) by analyzing diff hunks from unpatched code pieces. Kim et al. [23] replace occurrences of parameter variables, local variables, data types, and function calls with symbols to adapt the intermediate representation for type-2 clones. The abstracted code is then normalized by standardizing formatting elements such as comments, spaces, tabs, and line feeds, and converting all characters to lowercase. Similar normalization techniques are also applied in Woo et al.'s work [49, 51]. Woo et al. [50] also used the knowledge of the oldest vulnerable function to diversify the representation of vulnerable code.

Xiao et al. [54] employ a similar approach to abstraction and normalization but exclude format strings from abstraction due to their association with certain vulnerabilities. Additionally, Cui et al. [8] propose preserving identifiers using the K-means algorithm and manual selection of representative ones, such as `buf` and `len`, as they are often involved in vulnerabilities. Mirsky et al. [33] utilize LLVM to generate a customized LLVM IR, reducing syntactic-level differences. However, these methods compare similarity based on the equality of abstracted and normalized statements without considering their contextual semantic equivalence. Our approach addresses this by employing semantic equivalent statement matching for vulnerability signature matching.

7 CONCLUSIONS

In this paper, we focus on the vulnerabilities due to code reuse in OSS, known as vulnerable code clones (VCCs) or recurring vulnerabilities. We introduce VMUD, a novel approach for detecting "Vulnerabilities with Multiple Fixing Functions" (VM). VMUD selects the critical functions from VM for signature generation, which are a subset of the fixing functions. To deal with the potential decrease in recall due to excluding the remaining fixing functions, VMUD employs semantic equivalent statement matching. It aims to uncover more VM by duplicating signatures of the critical functions and match precisely by contextual semantic equivalent statement mapping on the duplicated signatures. Our evaluation has demonstrated that VMUD surpasses state-of-the-art vulnerability detection approaches by 17.6% in terms of F1-Score. VMUD has successfully detected 275 new VM from 84 projects, with 42 confirmed cases and 5 assigned CVE identifiers. We have released the source code and our evaluated data at our replication site [1].

REFERENCES

- [1] anonymous. 2024. *Replicating Material for VMUD*. Retrieved April 20, 2024 from <https://github.com/vmud20.github.io>
- [2] arcticwolf. 2024. *Beyond Sisen Navigating Rising Tide of Supply Chain Attacks*. Retrieved April 20, 2024 from <https://arcticwolf.com/resources/blog/beyond-sisense-navigating-rising-tide-of-supply-chain-attacks/>
- [3] businessinsights. 2024. *10 Stats on the State of Vulnerabilities and Exploits*. Retrieved April 20, 2024 from <https://www.bitdefender.com/blog/businessinsights/10-stats-on-the-state-of-vulnerabilities-and-exploits/>
- [4] Checkmarx. 2023. *9th Annual State of Software the Supply Chain*.
- [5] Checkmarx. 2024. *Checkmarx*. Retrieved April 20, 2024 from <https://checkmarx.com/>
- [6] Kenneth Ward Church. 2017. Word2Vec. *Natural Language Engineering* 23, 1 (2017), 155–162.
- [7] crowdstrike. 2024. *Supply Chain Attacks*. Retrieved April 20, 2024 from <https://www.crowdstrike.com/cybersecurity-101/cyberattacks/supply-chain-attacks/>
- [8] Lei Cui, Zhiyu Hao, Yang Jiao, Haiqiang Fei, and Xiaochun Yun. 2020. Vulddetector: Detecting vulnerabilities using weighted feature graph comparison. *IEEE Transactions on Information Forensics and Security* 16 (2020), 2004–2017.

- [9] doxygen. 2024. *Doxygen*. Retrieved April 20, 2024 from <https://www.doxygen.nl/>
- [10] Facebook. 2024. *Infer*. Retrieved April 20, 2024 from <https://fbinfer.com/>
- [11] Jiahao Fan, Yi Li, Shaohua Wang, and Tien N. Nguyen. 2020. A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries. In *Proceedings of the 17th International Conference on Mining Software Repositories*. 508–512.
- [12] gcc. 2024. *GCC*. Retrieved April 20, 2024 from <https://gcc.gnu.org/>
- [13] GitHub. 2023. *The 2023 State of the OCTOVERSE*. Retrieved April 20, 2024 from <https://octoverse.github.com/>
- [14] GitHub. 2024. *CodeQL*. Retrieved April 20, 2024 from <https://codeql.github.com/>
- [15] GitHub. 2024. *GitHub API Query*. Retrieved April 20, 2024 from https://api.github.com/search/repositories?q=language:C&order=desc&per_page=1000&page=1
- [16] gpac. 2024. *gpac repository*. Retrieved April 20, 2024 from <https://github.com/gpac/gpac/>
- [17] gpac. 2024. *Patch Commit for CVE-2021-32134*. Retrieved April 20, 2024 from <https://github.com/gpac/gpac/commit/328c6d682698fdb9878dbb4f282963d42c538c01>
- [18] Kim Herzig and Andreas Zeller. 2013. The impact of tangled code changes. In *Proceedings of the 10th Working Conference on Mining Software Repositories*. 121–130.
- [19] Ling Jiang, Hengchen Yuan, Qiyi Tang, Sen Nie, Shi Wu, and Yuqun Zhang. 2023. Third-Party Library Dependency for Large-Scale SCA in the C/C++ Ecosystem: How Far Are We?. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1383–1395.
- [20] Woosok Kang, Byoungso Son, and Kihong Heo. 2022. TRACER: signature-based static analysis for detecting recurring vulnerabilities. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 1695–1708.
- [21] Soheil Khodayari and Giancarlo Pellegrino. 2021. {JAW}: Studying Client-side {CSRF} with Hybrid Property Graphs and Declarative Traversals. In *Proceedings of the 30th USENIX Security Symposium*. 2525–2542.
- [22] Miryung Kim and David Notkin. 2009. Discovering and representing systematic code changes. In *Proceedings of the IEEE 31st International Conference on Software Engineering*. 309–319.
- [23] Seulbae Kim, Seunghoon Woo, Heejo Lee, and Hakjoo Oh. 2017. Vuddy: A scalable approach for vulnerable code clone discovery. In *2017 IEEE Symposium on Security and Privacy*. 595–614.
- [24] Rainer Koschke and Saman Bazrafshan. 2016. Software-clone rates in open-source programs written in C or C++. In *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, Vol. 3. 1–7.
- [25] Guanhua Li, Yijian Wu, Chanchal K Roy, Jun Sun, Xin Peng, Nanjie Zhan, Bin Hu, and Jingyi Ma. 2020. SAGA: efficient and large-scale detection of near-miss clones with GPU acceleration. In *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 272–283.
- [26] Guoren Li, Hang Zhang, Jinneng Zhou, Wenbo Shen, Yulei Sui, and Zhiyun Qian. 2023. A hybrid alias analysis and its application to global variable protection in the linux kernel. In *32nd USENIX Security Symposium*. 4211–4228.
- [27] Song Li, Mingqing Kang, Jianwei Hou, and Yinzhi Cao. 2022. Mining node.js vulnerabilities via object dependence graph and query. In *Proceedings of the 31st USENIX Security Symposium*. 143–160.
- [28] Zhen Li, Ning Wang, Deqing Zou, Yating Li, Ruqian Zhang, Shouhuai Xu, Chao Zhang, and Hai Jin. 2024. On the Effectiveness of Function-Level Vulnerability Detectors for Inter-Procedural Vulnerabilities. (2024), 1–12.
- [29] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Hanchao Qi, and Jie Hu. 2016. Vulpecker: an automated vulnerability detection system based on code similarity analysis. In *Proceedings of the 32nd annual conference on computer security applications*. 201–213.
- [30] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Yawei Zhu, and Zhaoxuan Chen. 2021. Syssev: A framework for using deep learning to detect software vulnerabilities. *IEEE Transactions on Dependable and Secure Computing* 19, 4 (2021), 2244–2258.
- [31] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, and Yuyi Zhong. 2018. Vuldeepecker: A deep learning-based system for vulnerability detection. *arXiv preprint arXiv:1801.01681* (2018).
- [32] microfocus. 2024. *Fortify*. Retrieved April 20, 2024 from <https://www.microfocus.com/en-us/cyberres/application-security>
- [33] Yisroel Mirsky, George Macon, Michael Brown, Carter Yagemann, Matthew Pruett, Evan Downing, Sukarno Mertoguno, and Wenke Lee. 2023. VulChecker: Graph-based Vulnerability Localization in Source Code. In *31st USENIX Security Symposium, Security 2022*.
- [34] NVD. 2024. *CVE-2021-32134 Detail4*. Retrieved April 20, 2024 from <https://nvd.nist.gov/vuln/detail/CVE-2021-32134>
- [35] nvd. 2024. *National Vulnerability Database*. Retrieved April 20, 2024 from <https://nvd.nist.gov/feeds/json/cve/1.1/>
- [36] Openstd. 2024. *Open Std*. Retrieved April 20, 2024 from <https://www.openstd.org/jtc1/sc22/wg14/www/docs/n1548.pdf>
- [37] Nam H Pham, Tung Thanh Nguyen, Hoan Anh Nguyen, and Tien N Nguyen. 2010. Detection of recurring software vulnerabilities. In *Proceedings of the 25th IEEE/ACM International Conference on Automated Software Engineering*. 447–456.
- [38] Rebecca Russell, Louis Kim, Lei Hamilton, Tomo Lazovich, Jacob Harer, Onur Ozdemir, Paul Ellingwood, and Marc McConley. 2018. Automated vulnerability detection in source code using deep representation learning. In *Proceedings of the 17th IEEE international conference on machine learning and applications*. IEEE, 757–762.
- [39] ShiftLeftSecurity. 2024. *Joern*. Retrieved April 20, 2024 from <https://github.com/ShiftLeftSecurity/joern>
- [40] Benjamin Steenhoek, Hongyang Gao, and Wei Le. 2024. Dataflow Analysis-Inspired Deep Learning for Efficient Vulnerability Detection. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.
- [41] Benjamin Steenhoek, Md Mahbubur Rahman, Richard Jiles, and Wei Le. 2023. An empirical study of deep learning models for vulnerability detection. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 2237–2248.
- [42] Friedrich Steimann, Philip Mayer, and Andreas Meißner. 2006. Decoupling classes with inferred interfaces. In *Proceedings of the 2006 ACM symposium on Applied computing*. 1404–1408.
- [43] Elco Visser. 2001. A survey of rewriting strategies in program transformation systems. *Electronic Notes in Theoretical Computer Science* 57 (2001), 109–143.
- [44] Shu Wang, Xinda Wang, Kun Sun, Sushil Jajodia, Haining Wang, and Qi Li. 2023. GraphSPD: Graph-based security patch detection with enriched code semantics. In *2023 IEEE Symposium on Security and Privacy*. 2409–2426.
- [45] Xin-Cheng Wen, Yupan Chen, Cuiyun Gao, Hongyu Zhang, Jie M Zhang, and Qing Liao. 2023. Vulnerability detection with graph simplification and enhanced graph representation learning. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering*. 2275–2286.
- [46] Seongil Wi, Sijae Woo, Joyce Jiyoung Whang, and Soeul Son. 2022. HiddenCPG: large-scale vulnerable clone detection using subgraph isomorphism of code property graphs. In *Proceedings of the ACM Web Conference 2022*. 755–766.
- [47] Wikipedia. 2024. *PageRank*. Retrieved April 20, 2024 from <https://en.wikipedia.org/wiki/PageRank>
- [48] wikipedia. 2024. *Precision and Recall*. Retrieved April 20, 2024 from https://en.wikipedia.org/wiki/Precision_and_recall
- [49] Seunghoon Woo, Eunjin Choi, Heejo Lee, and Hakjoo Oh. 2023. {VISCAN}: Discovering 1-day Vulnerabilities in Reused {C/C++} Open-source Software Components Using Code Classification Techniques. In *32nd USENIX Security Symposium*. 6541–6556.
- [50] Seunghoon Woo, Hyunji Hong, Eunjin Choi, and Heejo Lee. 2022. {MOVERY}: A Precise Approach for Modified Vulnerable Code Clone Discovery from Modified {Open-Source} Software Components. In *31st USENIX Security Symposium*. 3037–3053.
- [51] Seunghoon Woo, Dongwook Lee, Sunghan Park, Heejo Lee, and Sven Dietrich. 2021. {VoFinder}: Discovering the Correct Origin of Publicly Reported Software Vulnerabilities. In *30th USENIX Security Symposium*. 3041–3058.
- [52] Seunghoon Woo, Sunghan Park, Seulbae Kim, Heejo Lee, and Hakjoo Oh. 2021. CENTRIS: A precise and scalable approach for identifying modified open-source software reuse. In *2021 IEEE/ACM 43rd International Conference on Software Engineering*. 860–872.
- [53] Jiahui Wu, Zhengzi Xu, Wei Tang, Lyuye Zhang, Yueming Wu, Chengyue Liu, Kairan Sun, Lida Zhao, and Yang Liu. 2023. Ossfp: Precise and scalable c/c++ third-party library detection using fingerprinting functions. In *2023 IEEE/ACM 45th International Conference on Software Engineering*. 270–282.
- [54] Yang Xiao, Bihuan Chen, Chendong Yu, Zhengzi Xu, Zimu Yuan, Feng Li, Binghong Liu, Yang Liu, Wei Huo, Wei Zou, et al. 2020. {MVP}: Detecting Vulnerabilities using {Patch-Enhanced} Vulnerability Signatures. In *29th USENIX Security Symposium*. 1165–1182.
- [55] Can Yang, Zhengzi Xu, Hongxu Chen, Yang Liu, Xiaorui Gong, and Baoxu Liu. 2022. ModX: binary level partially imported third-party library detection via program modularization and semantic matching. In *Proceedings of the 44th International Conference on Software Engineering*. 1393–1405.
- [56] Annie TT Ying, Gail C Murphy, Raymond Ng, and Mark C Chu-Carroll. 2004. Predicting source code changes by mining change history. *IEEE transactions on Software Engineering* 30, 9 (2004), 574–586.
- [57] Junwei Zhang, Zhongxin Liu, Xing Hu, Xin Xia, and Shanping Li. 2023. Vulnerability detection by learning from syntax-based execution paths of code. *IEEE Transactions on Software Engineering* (2023).
- [58] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. *Advances in neural information processing systems* 32 (2019).

A EFFECTIVENESS RESULTS ON TARGET PROJECTS OF VMUD

We present true positives, false positives, detected by VMud and the false negatives of VMud in Table 9, The project identifier used in Table 9 corresponds to the project names listed in Table 10.

Table 9: The Number of True Positives, False positives, Detected by VMUD and the False Negatives VMUD Missed

PID.	TP (#)	FP (#)	FN (#)	Proj	TP (#)	FP (#)	FN (#)	Proj	TP (#)	FP (#)	FN (#)	Proj	TP (#)	FP (#)	FN (#)
1	25	2	4	27	1	0	0	53	2	0	3	79	1	0	0
2	4	0	0	28	3	0	0	54	1	0	0	80	1	0	0
3	26	2	5	29	1	0	0	55	2	0	0	81	1	0	0
4	4	0	2	30	2	0	0	56	1	0	0	82	1	0	0
5	1	1	0	31	1	0	0	57	0	1	0	83	1	0	0
6	20	1	7	32	1	0	0	58	0	1	0	84	1	0	0
7	4	0	1	33	1	0	0	59	1	0	0	85	1	0	0
8	6	0	0	34	8	1	0	60	1	0	0	86	1	0	0
9	3	1	1	35	1	0	0	61	6	0	1	87	1	0	0
10	3	0	0	36	0	1	0	62	1	0	0	88	0	1	0
11	1	0	0	37	17	0	4	63	1	0	0	89	0	1	0
12	2	0	1	38	0	8	0	64	0	4	0	90	1	0	0
13	4	1	0	39	2	0	0	65	2	0	0	91	1	0	0
14	8	0	3	40	1	0	0	66	2	0	0	92	1	0	0
15	10	1	6	41	2	0	0	67	0	3	0	93	0	1	0
16	3	0	0	42	0	8	0	68	1	1	0	94	1	0	0
17	5	1	5	43	1	0	0	69	0	1	0	95	1	0	0
18	2	0	0	44	1	0	0	70	0	1	0	96	1	0	0
19	2	0	0	45	2	0	3	71	1	0	0	97	0	0	1
20	0	1	1	46	1	0	0	72	0	1	0	98	1	0	0
21	1	0	0	47	1	0	0	73	1	0	0	99	1	0	0
22	4	0	0	48	1	0	0	74	1	1	0	100	0	1	0
23	1	1	0	49	1	0	0	75	1	0	0	101	1	0	0
24	3	0	1	50	1	0	0	76	2	0	0	102	0	0	1
25	1	0	0	51	23	1	4	77	1	0	0	103	1	0	0
26	2	0	0	52	8	1	0	78	1	1	0	104	1	0	0

Table 10: Projects ID (PID) and Project Names

PID	Project Name	PID	Project Name	PID	Project Name	PID	Project Name
1	hanwckf/rt-n56u	27	rogerclarkmelbourne/Arduino_STM32	53	huangz1990/annotated_redis_source	79	wireshark/wireshark
2	ravynsoft/ravynos	28	rmtheis/tess-two	54	OpenAtomFoundation/TencentOS-tiny	80	ejoy/ejoy2d
3	Stichting-MINIX-Research-Foundation/minix	29	eduard-permyakov/permafrost-engine	55	NetBSD/src	81	jart/cosmopolitan
4	seemoo-lab/nexmon	30	leixiaohua1020/simplest_ffmpeg_mobile	56	sdlpal/sdlpal	82	arendst/Tasmota
5	alibaba/tengine	31	peng-zhihui/Dummy-Robot	57	b4winckler/macvim	83	flipperdevices/flipperzero-firmware
6	fastos/fastsocket	32	armink/EasyLogger	58	veracrypt/VeraCrypt	84	nmap/nmap
7	panda-re/panda	33	Ralim/IronOS	59	openzfs/zfs	85	RedisGraph/RedisGraph
8	tanersener/mobile-ffmpeg	34	kbengine/kbengine	60	antirez/disque	86	roflor/proxychains-ng
9	premake/premake-core	35	endrazine/wcc	61	rizinorg/rizin	87	darktable-org/darktable
10	civetweb/civetweb	36	flatpak/flatpak	62	reactos/reactos	88	antirez/sds
11	madeye/proxydroid	37	AdAway/AdAway	63	nicolasff/webdis	89	openssl/openssl
12	y123456yz/reading-code-of-nginx-1.9.2	38	microsoft/WSL2-Linux-Kernel	64	FreeRDP/FreeRDP	90	mabeiianxi/small-video-record
13	freebsd/freebsd-src	39	catboost/catboost	65	radareorg/radare2	91	espressif/ESP8266_RTOS_SDK
14	CloverHackyColor/CloverBootloader	40	mridgers/clink	66	ApsaraDB/PolarDB-for-PostgreSQL	92	ctfs/write-ups-2016
15	damonkohler/sl4a	41	gpac/gpac	67	swaywm/sway	93	nginx/nginx
16	joncampbell123/dosbox-x	42	spotify/linux	68	mjolnirapp/mjolnir	94	jiangdongguo/AndroidUSBCamera
17	alibaba/AliOS-Things	43	emscripten-core/emscripten	69	greenplum-db/gpdb	95	ntop/PF_RING
18	zlgopen/awtk	44	Sunzxyong/Tiny	70	systemd/systemd	96	h2o/h2o
19	sumatrapdfreader/sumatrapdf	45	huangz1990/redis-3.0-annotated	71	RediSearch/RediSearch	97	gozfrees/gear-lib
20	nginx/nginx-releases	46	lavabit/magma	72	ultrajson/ultrajson	98	omnirom/android_bootable_recovery
21	peng-zhihui/ElectronBot	47	ossec/ossec-hids	73	slact/nchan	99	TeamWin/Team-Win-Recovery-Project
22	TelegramMessenger/Telegram-iOS	48	moby/hyperkit	74	ImageMagick/ImageMagick	100	coolwanglu/vim.js
23	Tencent/xLua	49	Proxmark/proxmark3	75	momotech/MLN	101	vifm/vifm
24	Cisco-Talos/pyrebox	50	cesanta/mongoose	76	nodemcu/nodemcu-firmware	102	matttrajca/sudo-touchid
25	aws/amazon-freertos	51	alibaba/LVS	77	saki4510t/UVCCamera	103	micropython/micropython
26	yangchaojiang/yjPlay	52	F-Stack/f-stack	78	libretro/RetroArch	104	InfiniTimeOrg/InfiniTime